

Contribution Technique

Atomic Item
Contribution

Pivot based

Contribution
at
end

with Data
Structure

①

2	3	5	1	2
---	---	---	---	---

Find the Sum of all the triplets in the array (i, j, k) such that

$$\sum_{1 \leq i \leq j \leq k \leq N} (arr[i] + arr[j] + arr[k])$$

Solⁿ $\rightarrow$ To find for every triplet, we have to find $(arr[i] + arr[j] + arr[k])$

Let's go the reverse way $\therefore$

for every element $arr[i]$, if we can find out, in how many triplet is it contributing, then my work will be over.

Now, each element will be present in $({}^{n-1}C_2)$ no. of triplets. simply because, leaving that element, I do need to

Pick the rest 2 elements from $(n-1)$ no. of elements.
 So, that's way $n-1C_2$.

$$\begin{aligned} \text{So, required Sum} &= \sum \text{arr}[i] * n-1C_2 \Rightarrow \text{const.} \\ &= n-1C_2 \left(\sum \text{arr}[i] \right) \Rightarrow \text{Sum of all the elements.} \end{aligned}$$

2. An inversion is defined as a pair of indices (i, j) where $(i < j)$ and $(\text{Arr}[i] > \text{Arr}[j])$.

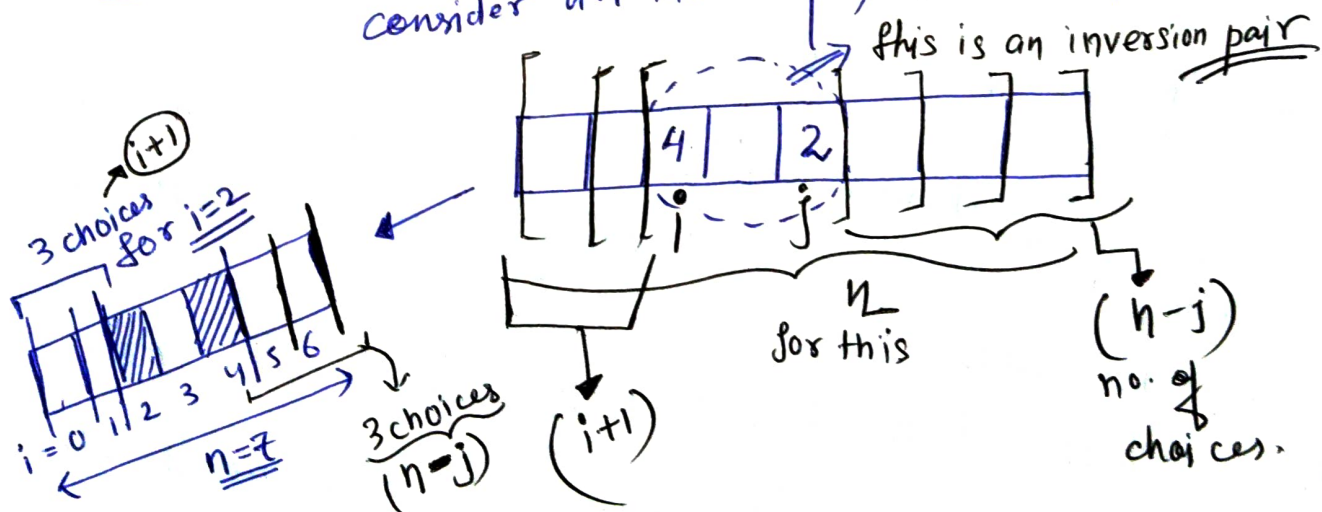
Given an array Arr of size N , calculate the Sum of inversion Counts of all the subarrays.

→ Brute force $\Rightarrow$ For every subarray, count the number of inversions of all the subarrays.

Let's reverse the idea

For every inversion pair, let us calculate in how many sub-array it is contributing.

Now, let us consider an inversion exist like below —



So, numbers of subarray it is contributing is $(i+1)*(n-j)$

↳ so, just for every inversion pair, i have to add its $(i+1)*(n-j)$.

So, code :-

```
sum = 0;
for (int i = 0; i < n; i++) {
    for (int j = i + 1; j < n; j++) {
        if (arr[i] > arr[j]) {
```

// means it is an inversion pair

// just add its contribution

$sum += (i+1)*(n-j);$

⇓
Contribution of that inversion pair.

```
        }
    }
}
```

● Contribution Technique by extending ends

③ Find the Sum of product of all the subarrays of the array.

ex :-

1	3	2
---	---	---

Product

1

→ 1

3

→ 3

2

→ 2

1

3

→ 3

3

2

→ 6

1

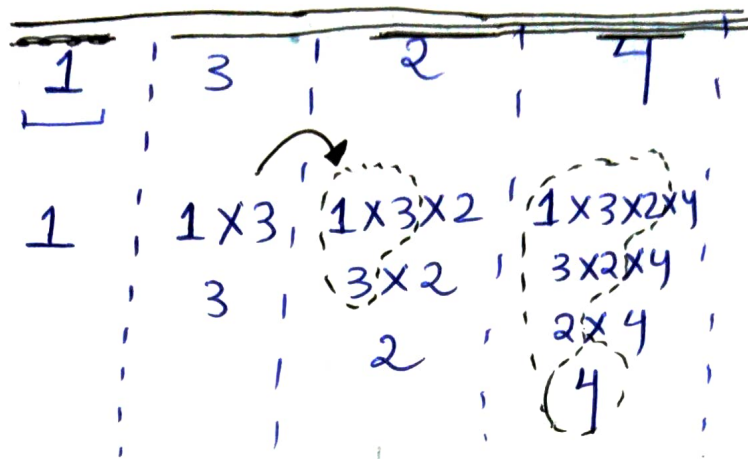
3

2

→ 6

(21) → Sum.

Let's see this idea of extending ends :-

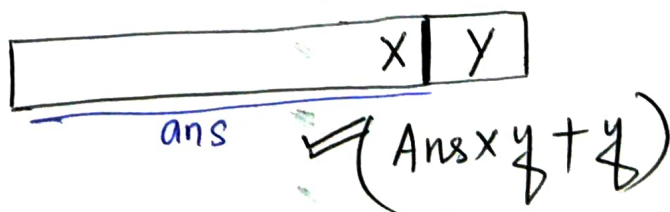


→ whatever be the sub-array, it will be any ^{one} of four types based on above

- ↳ ending at 1
- ↳ ending at 3
- ↳ ending at 2
- ↳ ending at 4

and if we have the sub-arrays ending at the previous index, to get the sub-arrays ^{ending} at this index, just we will have to add the present element to all the previous subarrays, and also one more sub-array starting and ending at this position.

In a box :-



Code:

```
int total = 0;
int curr-ans = 0;
for (int i = 0; i < n; i++) {
    curr-ans = curr-ans * arr[i] + arr[i];
    total += curr-ans;
}
cout << curr total << endl;
```

Here curr-ans is the SOP (Sum of Product) of all those sub-arrays ending at that particular index.

and total is the SOP of all those sub-arrays ending at this index and ending at previous indices.

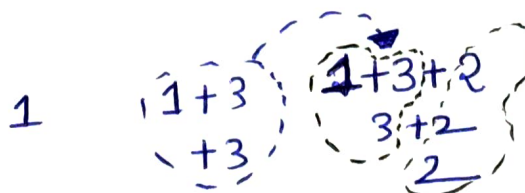
↳ Idea extension for SOS (Sum of Sum) of the Sub-arrays

ex:-

1

3

2



2 is getting added 3 times when $i=2$.

So, $(i+1)$.

So, in the Previous code, just replace curr-ans with

this

$$\text{curr-ans} = \text{curr-ans} + \text{arr}[i] * (i+1)$$

Core Idea

If I know my answer for the previous index, then can I extend it to find the answer for my next index??

⑤ Max^m Sum Subarray of an array

```
int global_
```

```
ans = 0;
```

```
int curr_end = 0;
```

```
for (int i = 0 ; i < n ; i++) {
```

```
    curr_end = max(curr_end + arr[i], arr[i]);
```

```
    global_ans = max(global_ans, curr_end);
```

```
}
```

```
cout << global_ans << endl;
```

Remember If empty sub-array is allowed then global_ans will be initiated with zero.

If empty sub-array is not allowed, then global_ans will be initiated with INF_MIN, then only we will be able to consider a condition if the maximum sum is also a negative number.

Idea for this Problem: $\rightarrow$ Similar to previous qⁿ of SOS, from the previous index, we extrapolate and find the sum of the subarrays ending at that index, (similar to previous), and then at curr_end we take the

max of extrapolated answers and the sub-array starting and ending at that index. Then updating the answer at the global level.

→ Similar to SOS, SOP, we can POP as well, but POS can't be done.